1. Executive Summary

What the system is

The GTM Lead Intelligence Agent is an internal tool that finds enterprise buying opportunities in public news and press releases and turns them into scored, region-filtered leads for Trianz sales teams. It is one Next.js application that does everything: it builds searches, collects articles, asks an AI model to judge each one, saves the qualified leads, and emails a digest. Its data lives in PostgreSQL.

How a user experiences it

A user signs in with their Trianz Microsoft account, selects the regions, industries, and Concierto offerings they care about, and sets a schedule. From then on the system runs discovery on that schedule (or on demand). The user opens the portal to see scored leads with a summary, the reason each one qualified, the stage of the buyer's project, and a link to the source article. A digest email summarizes new leads after each run.

What makes it work

Three design choices carry most of the weight:

- **Signals drive the search.** Buying phrases such as "data center exit" are stored per offering and turned into news queries. Sales owners can change what the system looks for without a code change.
- **Two checks after the AI.** The model is asked to respect the user's offerings and regions, but the code checks both again before saving. The code check is the one that counts.
- **One process.** Scheduling, discovery, AI calls, and the API all run in one Node process. This keeps the system simple to deploy, and it is also the source of its main operational limits, described in section 10.

Status at the time of writing

The core flow is built: sign-in, preferences, signals, discovery across three news sources, AI scoring with a fallback, lead storage, dashboard counts, scheduled runs, and the digest. The main open work is operational hardening: durable scheduling, delivery tracking for email, access control for signal changes, and documentation that matches the code. Section 23 lists the work.

2. Overview and Problem

The problem

Sales teams find new opportunities by reading news. The useful signal is spread across press releases, trade publications, and regional news. A data center exit, a new FinOps program, or an AI initiative usually appears days before an account team hears about it, and only if someone is reading the right feed at the right time.

Manual monitoring has four specific problems:

- **Coverage.** One person cannot read every region, industry, and source every day.
- **Consistency.** Two reps watching the same market will judge the same article differently.
- **Timing.** A signal found a week late is often a lost opportunity.
- **Repetition.** The same press release is reposted across sites, so people read it several times.

What the agent does about each problem

Problem	What the agent does
Coverage	Runs scheduled searches for each user's regions, industries, and offerings, across three sources.
Consistency	Applies the same prompt, the same score bands, and the same offering and region rules to every article.
Timing	Runs on a schedule, and the digest tells users when new leads arrive.
Repetition	Removes duplicates within a run and against stored articles, by URL and by content hash.

What the agent does not do

- It does not contact customers or write outreach.
- It does not verify that a company is a real prospect beyond what the article says.
- It does not track which leads a rep acted on. There is no pipeline stage or follow-up field.
- It does not read paywalled full articles. It works from the title and the snippet that the news feed provides.

Who uses it

User	What they need	How they use it
Sales reps and account teams	Leads in their territories, with enough context to act	Open the portal, review new leads, open source articles, and follow up outside the system.
Sales leadership	A view of the pipeline by score and by offering	Read the dashboard counts and the digest, and ask for changes to signals or coverage.
Offering owners	Control over what the system searches for	Maintain the buying signals for their offering. Currently done through the API or migrations.
Platform team	A stable, observable service	Deploy, run migrations, watch startup logs, and manage the Azure resources.

What a lead contains

Each lead is one opportunity from one article, for one user. It includes:

- **Who:** the company and its initials, industry, and country.
- **What:** the buying signal detected, and the Concierto offering it maps to.
- **How strong:** a score label (Very Hot, Hot, or Warm), a score from 0 to 100, and the reason it qualified.
- **Context:** a one-line summary, the project stage, the business impact, and the procurement activity, as the model read them from the article.
- **Source:** the publisher, the article link, and the published and detected dates.

3. Technology Stack

Application

Layer	Technology	Why it is here
Web framework	Next.js 16.3 with the App Router;	Pages and API route handlers in one

	Turbopack in development	project; server code runs in the same process as the scheduler.
Language	TypeScript	Shared types between the API, services, and portal.
UI	React 19, Tailwind CSS 4, shadcn-style components	The portal screens.
Authentication	NextAuth v4 with the Azure AD provider	Microsoft sign-in for Trianz staff, with no separate password store.
Database client	pg connection pool	Direct SQL, with the pool size and timeouts set from environment variables.

Data and integrations

Layer	Technology	Why it is here
Database	PostgreSQL 16 (Azure Database for PostgreSQL in production; `postgres:16-alpine` in Docker Compose)	Relational storage with arrays, check constraints, and advisory locks.
AI analysis	Azure OpenAI, gpt-4o deployment	Reads an article and returns structured lead data in JSON.
Fallback analysis	Local keyword analyzer	Lets development and tests run without Azure credentials.
News sources	Google News RSS, PR Newswire RSS, Business Wire RSS	Article collection with no paid data feed.
Email	Azure Communication Services Email SDK	Sends the digest from the application.
Digest feed	`GET /api/digest` (HTML)	Lets Power Automate forward the digest without the app sending it.

Hosting and operations

Layer	Technology	Why it is here
Production hosting	Azure App Service, Linux, Node runtime	Managed hosting with App Service configuration for environment variables.
Local containers	Docker Compose, with one PostgreSQL service	A local database without installing PostgreSQL by hand.
Scheduling	In-process timer, started from `instrumentation.ts`	No separate scheduler service to run.
Tests	Node test runner through `tsx --test`	No test framework to install beyond the project's own tooling.

Version notes

- Next.js is pinned to 16.3.3 in the lockfile.
- next-auth is at 4.24.x, which is the v4 line. This is not Microsoft's "Easy Auth" feature; the sign-in is configured in application code.
- `docker-compose.yml` defines only PostgreSQL. The application runs with `npm run dev` or `npm start`, not inside a container.

4. System Architecture

Pipeline diagram



Components and where they live

Component	File or folder	Responsibility
Startup hook	`instrumentation.ts`	Runs migrations, then starts the scheduler.
Scheduler	`lib/services/scheduler-service.ts`	Decides which users are due and starts their runs.
Discovery service	`lib/services/discovery-service.ts`	Orchestrates one run from start to finish.
Query generation	`lib/services/query-`	Turns signals and preferences into

	`generation-service.ts`	search strings.
News sources	`lib/discovery/sources/`	One adapter per source.
Deduplication	`lib/services/deduplication-service.ts`	URL normalization, content hashing, duplicate checks.
AI client	`lib/ai/azure-openai-client.ts`	The model prompt, the call, the response parsing, and the fallback analyzer.
Lead service	`lib/services/lead-generation-service.ts`	Applies the barriers and saves leads.
Region rules	`lib/domain/regions.ts`	Maps countries to regions and checks membership.
Email	`lib/services/email-service.ts`	Sends the digest through Azure Communication Services.
Auth	`lib/auth/auth-options.ts`, `proxy.ts`	Sign-in, sessions, and route protection.
Current user	`lib/workspace/current-user.ts`	Resolves the signed-in user for each API call.
Migrations	`lib/db/migrate.ts`, `db/migrations/`	Versioned schema changes.

How the pieces connect

- **Startup.** When the Node runtime starts, `instrumentation.ts` runs the migration runner. If that succeeds or fails, it then starts the scheduler. A migration failure is logged and the server still starts.
- **Request path.** A browser or API client calls a route. `proxy.ts` checks the NextAuth session first. The route handler resolves the user from the session, validates input, calls a service, and returns a response envelope.
- **Run path.** A run, whether scheduled or manual, goes through the discovery service. That service calls the sources, the deduplication service, the lead service, and the email service in that order.
- **Data path.** Services call repositories. Repositories are the only code that writes SQL for each table.
- **Failure isolation.** Each source, query, and article is wrapped separately. One failure is logged and the run continues. Only a failure outside those loops fails the whole run.

Why this shape

A single process is the simplest thing that works for an internal tool with a small user base. The trade-off is that the scheduler and the web server share a lifetime. Section 10 covers the consequences.

5. User Roles and Permissions

Current model

There is one role: `Trianz employee`. Every user who signs in with a Trianz Azure AD account is created with that role. The role is stored on the user row and is returned by the API, but no code checks it. Access is controlled by ownership, not by role.

What each signed-in user can do

Action	Allowed	How it is enforced
View their own leads, runs, and dashboard	Yes	Every query is filtered by the session user's ID.
Change their own preferences and schedule	Yes	The preference row is looked up by the session user's ID.

Start a discovery run for themselves	Yes	`POST /api/discover-leads` runs for the session user only.
View another user's leads	No	No route accepts a user ID from the request.
Add or change a buying signal	Yes, for anyone	Signals are global. `POST /api/signals` has no owner check.
Trigger scheduled runs for all users	Yes, for anyone signed in	`POST /api/scheduler?force=true` runs every scheduled user.
Change another user's role or name	No	`PATCH /api/workspace` changes only the session user.

What this means in practice

- **Leads are private to each user.** A rep cannot see another rep's leads through the portal or the API.
- **Signals are shared.** A signal added by one user changes the searches for every user who has that offering. This is the most important permissions gap in the system.
- **Scheduler triggers are shared.** Any signed-in user can run the forced scheduler, which starts discovery for every scheduled user at once.

Recommended model

If the system grows beyond the current group, the minimum change is:

- **Admin** role, set by an allow-list of Azure AD users, allowed to create, change, and delete signals and to call the scheduler routes.
- **Member** role, the current `Trianz employee` behavior, for viewing and managing their own data.

This is listed in section 23. It is not built, and the role field is not yet used for anything.

6. Authentication and Authorization

Sign-in flow

1. An unsigned user opens any page and is redirected to `/auth/signin`.
2. The user chooses to sign in with Microsoft. NextAuth redirects them to Azure AD.
3. Azure AD returns the user to `/api/auth/callback/azure-ad`.
4. The sign-in callback creates or updates the `users` row by email, with the role `Trianz employee`.
5. If the user has no preferences row, the callback creates defaults: regions APAC, industries All industries, offering Concierto Migrate, notification email set to the user's email, schedule 01:30 daily, and scheduled discovery enabled.
6. The JWT callback stores the database user ID in the token. The session callback copies it into `session.user.id`.

The 01:30 default is local server time. On Azure App Service the server runs in UTC, so 01:30 is 07:00 IST. A user in a different time zone will get a run at a different local hour.

Session model

- Sessions are JWT-based. There is no session table.

- Because the user ID is in the token, a session stays valid until the token expires, even if the user row is changed.
- If a session refers to a user ID that no longer exists, route handlers return `USER_NOT_FOUND`.

Route protection

`proxy.ts` applies NextAuth's `withAuth` to every path except:

- `/api/auth/*`, the NextAuth endpoints themselves
- `/auth/signin`, the sign-in page
- `/_next/static/*`, `/_next/image/*`, and `favicon.ico`

For pages, an unauthenticated request is redirected to sign-in. For API routes, the route handler's call to `getCurrentWorkspaceUser()` returns `401 UNAUTHORIZED` if there is no session user.

Per-user scoping

Route handlers never accept a user ID from the client. They call `getCurrentWorkspaceUser()`, which reads the session and loads the user. Every repository call that follows uses that ID. This is the main protection against one user reading another user's data.

Configuration the sign-in depends on

Setting	Effect if missing
`AZURE_AD_CLIENT_ID`, `AZURE_AD_CLIENT_SECRET`, `AZURE_AD_TENANT_ID`	The provider is created with empty values, and sign-in fails at the Microsoft step. Missing values are not caught at startup.
`NEXTAUTH_SECRET`	NextAuth cannot sign tokens securely in production.
`NEXTAUTH_URL`	Callback URLs may be built from the wrong host, which breaks sign-in behind App Service.

Known gaps

- `NEXTAUTH_URL` and `NEXTAUTH_SECRET` are missing from `.env.example`, so a new developer may not know they are needed.
- The Azure AD values are read with non-null assertions in `lib/auth/auth-options.ts`, so a misconfiguration is discovered on the first sign-in attempt.
- Scheduler and digest routes depend on the same session. A non-browser caller such as Power Automate needs a valid session cookie, and the repo has no service-to-service credential. Confirm how external callers will authenticate before wiring up a schedule.

7. Core Workflows

7.1 Discovery run (manual)

7. The user clicks the discovery action in the portal, which calls `POST /api/discover-leads`.
8. The route resolves the user and calls the discovery service for that user only.
9. The run goes through the full pipeline (section 8) and returns the run summary.
10. The user sees the new leads on the dashboard and in the explorer.

A manual run skips the scheduled-discovery-enabled check. It still requires preferences to exist.

7.2 Discovery run (scheduled)

11. The scheduler tick finds the user is due (section 10).
12. The scheduler calls the same discovery service, with the manual flag off.
13. If the user has disabled scheduled discovery, the run is rejected before any work starts.
14. The run is otherwise identical to a manual run.

7.3 Reviewing leads

- **Dashboard.** Shows lead counts by score label, the five most recent leads, the latest run, and the scheduling status.
- **Lead explorer.** Lists leads with a search box that matches company or signal, and filters for industry, country, offering, and score. Results are paged, with a limit of 1 to 100 per page.
- **Lead detail.** Shows the full lead and a link to the source article. A lead belonging to another user returns `LEAD_NOT_FOUND`, the same as a missing lead, so one user cannot test whether another user's lead IDs exist.

7.4 Maintaining preferences

- The user saves regions, industries, offerings, the notification email, the schedule time, the frequency, the schedule day, the enabled flag, and the workspace name through `POST /api/preferences`.
- Saving sets `updated_at`, which resets the scheduler's last-run baseline (section 10.4). The next matching slot runs, regardless of earlier runs.

7.5 Maintaining signals

- Signals are added or updated with `POST /api/signals`, which upserts by offering and signal text.
- Signals are read with `GET /api/signals`. With offering filters, only active signals for those offerings are returned, sorted by priority.
- Because signals are global, changes affect every user with that offering (section 5).

7.6 Digest

- After a run that produced leads, the discovery service asks the email service to send the digest to the user's notification address.
- The same digest content is available as HTML from `GET /api/digest`, for Power Automate to forward on its own schedule.

7.7 Health and status

- `GET /api/healthz` returns a liveness response for the App Service health probe.
- `GET /api/scheduler` shows whether the timer is running and what the last tick and last run key were. It also starts the timer if it is not running, so calling it is not purely read-only.

8. Pipeline, Step by Step

This section follows one discovery run from start to finish, using the code paths in `discovery-service.ts`.

Step 1: Load the run's inputs

- The service loads the user's preferences. If none exist, the run fails with a clear message.

- If the run is scheduled and scheduled discovery is off, the run fails before any work.
- The service loads active signals for the user's offerings, sorted by priority.
- The sources list is set to Google News, PR Newswire, and Business Wire, unless a caller passes a different list.

Step 2: Generate queries

Query generation is in `lib/services/query-generation-service.ts`. For each signal, four strategies run in order:

15. `"{signal} {industry} enterprise {geo}"`, for up to two of the user's industries and up to two of the user's regions. This is the most specific query.
16. `"{signal} {first industry} enterprise"`.
17. `"{signal} enterprise {geo}"`, using the first region.
18. `"{signal} enterprise"`, a global sweep with no industry or region.

The geo text comes from the region. For example, EMEA becomes `Europe OR Middle East OR UK OR Germany OR UAE`, and North America becomes `United States OR Canada`. Duplicate query text is removed, and the list is capped at 10 queries for runs started from discovery.

Fallback queries. If the user has no active signals for their offerings, the discovery service uses a fixed list of queries for each offering. These are hard-coded in `discovery-service.ts`, for example "enterprise cloud migration" for Concierto Migrate and "FinOps cloud cost optimization enterprise" for Concierto Manage. The fallback is a safety net, not a replacement for signals.

Each query records the offering it came from, so the lead service can later check the offering.

Step 3: Fetch articles

- The service loops over queries, and for each query it loops over sources. Each source call runs one after another, not in parallel.
- A failed source call is logged as a warning and skipped. The loop continues.
- Every article returned is added to one collection for the run.

With 10 queries and 3 sources, a run makes up to 30 source calls. This is the first place a run can be slow.

Step 4: Normalize and hash

- Each article's canonical URL is normalized: the host is lowercased, default ports are removed, tracking parameters are removed, remaining parameters are sorted, and the fragment is dropped.
- The content hash is the SHA-256 of the normalized title, the normalized canonical URL, and the first 250 characters of the normalized snippet. Title and snippet are lowercased and have whitespace collapsed first.

Two copies of the same press release, with different tracking parameters, produce the same hash.

Step 5: Deduplicate

`deduplicateArticles` checks each article in order:

19. **Within this run.** If the hash or the canonical URL has already appeared earlier in the collection, the article is a duplicate.
20. **In the database by hash.** `articles.content_hash` is unique, so a match means the article was stored in an earlier run.

21. **In the database by URL.** A match on `canonical_url` also counts as a duplicate.

The result is the list of unique articles and a count of duplicates removed.

Step 6: Store new articles

Each unique article is upserted into `articles` with a link to this run. Storing happens before analysis, so an article is recorded even if the AI call later fails.

Reuse when nothing is new. If no new articles were stored but the run did collect articles, the service looks up up to 40 of the collected articles in the database and analyzes those against the current preferences. This is how a preference change produces leads without waiting for new news. Only the first 40 collected items are checked for reuse, so a run with a lot of old matching news may not reuse all of it.

Step 7: Analyze each article

- Each article is sent to `azureOpenAiClient.analyzeArticle`, one at a time.
- With Azure OpenAI configured, the call is a chat completion to the gpt-4o deployment in JSON mode, with temperature 0.1 and a 25-second timeout.
- The prompt includes the five Concierto offerings, the customer pains each one addresses, the problem states, the score bands, and the disqualification rules. It asks the model to identify the buyer, the buying signal, the country, and the offering.
- The response must be a JSON object. The fields are listed in section 13.4.
- Problem state C means the buyer has already solved the problem. The model is asked to redirect to the next-phase offering, lower the score by 10, and cap the score at Warm.
- Without Azure configuration, the keyword analyzer runs instead. It picks an offering and signal from keywords, guesses the industry and country, and assigns a score from a few trigger words. It is a development tool. It cannot separate a buyer from a vendor reliably.

If an article's analysis throws, such as on a timeout, the error is logged and the article is skipped. The run continues.

Step 8: Apply the barriers and save leads

Covered in detail in section 11. In order:

22. The analysis must say the article is a valid opportunity, and the industry must be validated.
23. The offering must be one of the user's offerings.
24. The country must fall in one of the user's regions.
25. The company and offering pair must not already be a lead for this user, either earlier in this run or among the user's most recent 1,000 leads.

Accepted leads are upserted into `leads`, keyed by user and content hash. The company initials are computed from the first letters of the first two words of the company name.

Step 9: Complete the run record

- On success, `search_history` is marked `completed` with the articles found, duplicates removed, and leads generated.
- On failure, `search_history` is marked `failed` with the error message, and the error is re-thrown to the caller.

Step 10: Send the digest

- If the user has a notification email, the service calls the email service without waiting for the result.
- The email service skips the send if there are no leads, the connection string is missing, or the address is invalid.
- A send failure is logged. It does not change the run's result, and it is not recorded anywhere else.

Outputs of a run

Output	Where it goes
Run summary	`search_history`: status, times, articles found, duplicates removed, leads generated, queries, sources
Articles	`articles`, linked to the run
Leads	`leads`, visible in the dashboard and explorer
Digest	Email, plus GET `/api/digest` for Power Automate

Sub-sections

Query Generation

Covered in Step 2. The query list is also stored on the run record, so a run can be explained after the fact.

Discovery and Deduplication

Covered in Steps 3 to 6. Deduplication happens twice: once against the current batch and once against stored articles. Articles are stored across all users, so a press release found by one user is recorded once, and each user's lead for it is evaluated separately.

AI Analysis and Scoring

Covered in Step 7. Scores are labeled Very Hot (85 to 100), Hot (70 to 84), or Warm (50 to 69). The model returns both the label and the number, and the code stores both.

Lead Service, Isolation, and Persistence

Covered in Step 8 and section 11. The lead service loads up to 1,000 of the user's existing leads at the start of each article batch and builds a set of company and offering keys. That set is the duplicate check for the whole run.

Search History and Dashboard Counts

Covered in Step 9. Dashboard counts come from the leads table, and the run list comes from search history. A run that is still `running` shows in the list but has no final counts. A run that crashed before completing stays `running` only if the process died before the failure handler ran; a clean error marks it `failed`.

9. Discovery Sources

The three sources

Source	Adapter	How it is queried	Notes
Google News	`lib/discovery/sources/google-news.ts`	The Google News RSS search endpoint, with a 10-second timeout	Titles are often "Headline - Publisher". The adapter splits off the publisher. Snippets are cut to 500 characters.

PR Newswire	`lib/discovery/sources/pr-newswire.ts`	RSS feed	Press releases, which often name a company and a deal directly.
Business Wire	`lib/discovery/sources/business-wire.ts`	RSS feed	Press releases. Overlaps with PR Newswire on some announcements.

Parsing

- The RSS parser is written by hand as regular expressions over `<item>` blocks. It does not use an XML library.
- Each item's title, link, description, date, and source are extracted. When a source tag is missing, the publisher is taken from the suffix of the title.
- A source that fails, or returns something unparseable, contributes no articles. The run does not stop.

Source types

The source name is a fixed union type in `lib/discovery/sources/types.ts`: `google_news`, `pr_newswire`, or `business_wire`. Adding a source means extending this type as well (section 19).

Settings that look configurable but are not

`RSS_SOURCES` and `GOOGLE_NEWS_LANGUAGE` appear in `.env.example`, but the code does not read them. The source list is set in `discovery-service.ts`, and the Google News URL hard-codes the English US locale (`hl=en-US`, `gl=US`, `ceid=US:en`). Changing either needs a code change.

Coverage limits

- Google News aggregates many publishers, so a single article can appear through several of them. Deduplication by canonical URL catches only the exact same link. Different publishers covering the same event are kept as separate articles, and the lead service then de-duplicates by company and offering.
- Press release feeds carry the publisher's own wording, which is usually promotional. The model is asked to judge whether a company is a buyer, not a vendor, but a press release from a consultancy can still pass as a buyer story.
- The system has no paywall access. Snippets come from the feed, so an article whose key facts appear only in the body will be judged on less information.

10. Scheduler Internals

What the scheduler is

`lib/services/scheduler-service.ts` is an in-process timer. It is not a cron job, a queue, a WebJob, or a separate worker. It exists only while the Node process exists.

Lifecycle

- **Start.** `instrumentation.ts` calls the start function once the migrations have finished, whether or not they succeeded. Starting twice does nothing.
- **First check.** Five seconds after start, one check runs. This catches runs that were due while the process was starting.
- **Regular ticks.** A check runs every 60 seconds.

- **Process lifetime.** Both timers are unreferenced, so they do not keep Node alive on their own. In practice the web server keeps the process alive.
- **Stop.** The stop function clears both timers. Nothing calls it in normal operation.

What each tick does

26. Records the tick time, for the status endpoint.
27. Loads every preference row where scheduled discovery is enabled.
28. For each user, computes the target time as the first five characters of the saved schedule time.
29. Compares the target time with the server's local clock, formatted as `HH:mm`. Users whose target differs are skipped. The check runs every minute, so a 01:30 user is evaluated during the 01:30 tick.
30. Builds the run key as user ID, date, and target time, for example `<user-id>:2026-10-15:01:30`.
31. If the key is already in the in-memory set, the user is marked "already ran" and skipped.
32. Checks the frequency rule (section 10.3). If it is not due, the user is skipped.
33. Adds the key to the in-memory set, then starts the discovery run for that user.
34. Errors are caught per user. One user's failure does not stop the other users in the same tick.

Frequency rules

Frequency	When it runs
Daily	Every day at the target time.
Weekly	On the scheduled weekday, and at least 7 days after the last completed run.
Biweekly	On the scheduled weekday, and at least 14 days after the last completed run.

- The scheduled weekday uses `0` for Sunday through `6` for Saturday. The default is `1` (Monday).
- A weekly or biweekly user with no earlier completed run runs on the first matching weekday.
- The "days since last run" count uses whole days, rounded down.

Baseline reset on preference change

The last-run lookup counts only completed runs that started after the user's preferences row was last updated. Saving new preferences therefore resets the cadence.

Example: a user on a weekly schedule ran on Monday. On Wednesday they change the schedule to daily at 07:00. The next run is at 07:00 that day, not on the following Monday. Without this reset, the old run would hold the user to the old schedule.

Fail-open

If the lookup for the last run fails, for example because the database is briefly unavailable, the run goes ahead. The reasoning is that a silent skip would be harder to notice than an extra run. The cost is that a weekly user can receive an extra run during an outage.

The duplicate-run lock and its limit

- The lock is an in-memory set of run keys. It stops a second start in the same minute.
- It is never cleared, so it grows by one key per scheduled user per day. The growth is small for an internal user base, but it is unbounded.
- It does not survive a restart. A restart within the scheduled minute can start the same run again.

- It does not coordinate across instances. Each App Service instance has its own set and its own timer.

Limitations

Limitation	Effect	Mitigation today
Timer runs only while the process is up	Scheduled runs stop during downtime. Missed runs are not replayed.	Keep the service running; watch the health probe.
Duplicate lock is in memory	A restart in the scheduled minute can double a run.	The window is one minute; accept the risk.
Multiple instances run multiple timers	Each instance can start the same run.	Run one instance.
Server local time	Schedule times are interpreted in the server's time zone.	Azure App Service is UTC, so schedule times are UTC.
Run is not queued	If the server is busy, a run starts late rather than waiting in a queue.	None.

External trigger

Call	What it does
`GET /api/scheduler`	Returns status: whether the timer is running, the last tick, the last run key, and the active schedules. Also starts the timer if it is stopped.
`POST /api/scheduler`	Runs one check now, with the same time and frequency rules. Returns each user's status: triggered, skipped, or already ran.
`POST /api/scheduler?force=true`	Runs discovery for every scheduled user immediately. Ignores time and frequency. Intended for testing and recovery.

All three require a session. A caller that is not a browser must present a valid session. Confirm the external caller's authentication before using these routes from Power Automate.

11. Isolation Rules

Isolation means a user only receives leads that match what they chose. It is enforced twice: once in the prompt, which asks the model to respect the user's choices, and once in code after the model replies. The code check is the authority. The model can ignore a prompt, and the code cannot.

11.1 Offering barrier

The five offerings

Offering	Focus
Concierto Migrate	Moving on-premises or legacy infrastructure to the cloud.
Concierto Modernize	Re-architecting applications for cloud-native operation.
Concierto Manage	Managed cloud operations, FinOps, governance, and reliability.
Concierto Maximize	Getting measurable value and return from cloud investment.
Concierto Intelligence	AI, data, and analytics transformation.

The rule

A lead is saved only if its offering is in the user's offerings. If the user's offerings list is empty, every offering passes. The default preferences always set at least one offering, so this case only occurs if the data is changed directly.

Redirects

The prompt tells the model that a buyer who has already solved the problem (state C) should be redirected to the next-phase offering. For example, a completed migration is redirected to Modernize or Manage. The redirected offering is then checked against the user's offerings like any other. If the user did not choose the redirect target, the lead is rejected.

Limits

- Offering names are a fixed set in code. A signal that uses a name outside the five can never produce a lead.
- The barrier cannot correct a wrong offering. If the model picks Migrate for an article that is really about Manage, and the user selected only Manage, the article is rejected even though it was relevant.

11.2 Region barrier

The region check lives in `lib/domain/regions.ts`.

The algorithm

35. If the user's regions list is empty, or contains Global, every country passes.
36. Otherwise, the country name is mapped to a region:

    - First, an exact, case-insensitive match against the country lists.
    - Second, keyword matches for common variants and city names, for example a city name or a short form of a country name.
    - If nothing matches, the country maps to Global.

37. The country passes if its region is one of the user's regions.

Regions and countries

Region	Countries on the exact list
North America	United States, Canada, Mexico
APAC	Singapore, Japan, Australia, India, China, Hong Kong, New Zealand, South Korea, Indonesia, Malaysia, Philippines, Thailand, Taiwan, Vietnam
EMEA	United Kingdom, Germany, France, Netherlands, Switzerland, Spain, Italy, Sweden, Ireland, UAE, Saudi Arabia, Poland, Belgium, Norway, Denmark, Finland, Austria, South Africa
Global	Matches every region. It is a choice a user makes, not a country.

Worked examples

Article country (from model)	Maps to	User regions	Result
United States	North America	APAC	Rejected
United Kingdom	EMEA	APAC, EMEA	Kept
Sydney	APAC (keyword)	APAC	Kept

Brazil	Global (no match)	APAC	Rejected
Brazil	Global (no match)	Global	Kept

The Brazil row shows the main risk. Any country the list does not know is treated as Global, which passes only for users who chose Global. A real South American opportunity is therefore lost for everyone else until the country is added.

Changing the rules

- To add a country, add it to the right list in `REGION_COUNTRIES`.
- To handle a spelling the model may return, add a keyword to the keyword matching.
- Add a test for the new country, showing it is kept for the right region and rejected for the others.

11.3 Industry validation

The model returns a flag that says whether the company is in a valid industry. A saved lead must have this flag set. The industry list the user chose is passed to the model, but the code does not filter the industry name against the user's list after the model replies. A user who chose Manufacturing and receives a lead the model tagged as Manufacturing will see it. A lead tagged with an industry the user did not choose, but with the flag set, is saved as well. Tightening this is listed in section 23.

11.4 Rejection behavior

What happens to a rejected article:

Stage	Result
Model says not a valid opportunity	Article skipped. No lead.
Industry not validated	Article skipped. No lead.
Offering not selected	Article skipped. No lead.
Region not allowed	Article skipped. No lead.
Company and offering already a lead	Article skipped, and a console line is written.
Analysis throws	Article skipped, and a console warning is written.

In every case:

- No row is written to `leads`.
- The article row stays in `articles`, so it is not treated as new on the next run.
- The article is still counted in the run's articles-found total.
- No reason is stored. The model's reason field is kept only for saved leads.

Investigating a missing lead

38. Find the article in `articles` by title or canonical URL.
39. Check the run in `search_history` for the query that found it.
40. Re-run the analysis for that article against the user's preferences, and compare the model output with the rules above.
41. If the model output looks correct and the rule is wrong, fix the rule. If the model output is wrong, the prompt needs work.

Because rejections are not stored, step 3 is the only way to explain a rejection after the fact. Section 23 proposes storing a reason for each rejection.

12. Data Model

Tables

Table	Row meaning	Written by	Read by
`users`	One signed-in person	Sign-in callback, workspace PATCH	Session lookup, every route
`preferences`	One set of settings per user	Sign-in defaults, POST `/api/preferences`	Discovery, scheduler, lead service
`offering_signals`	One buying phrase for one offering	POST `/api/signals`, migrations	Query generation
`search_history`	One discovery run	Discovery service	Dashboard, run list, scheduler
`articles`	One unique article	Discovery service	Deduplication, reuse path
`leads`	One opportunity for one user	Lead service	Dashboard, explorer, digest
`schema_migrations`	One applied migration	Migration runner	Migration runner

Relationships

- A user has one preference row.
- A user has many runs, leads, and (through runs) articles.
- An article can have many leads, one per user who matched it.
- A lead points to its article and its run. If either is deleted, the link is cleared and the lead is kept.
- Signals are not linked to users. They belong to an offering.

Migrations

Migrations are SQL files in `db/migrations/`, applied in filename order. Each file runs in a transaction.

File	Change	Why it matters
`001_initial_schema.sql`	Creates the six core tables, indexes, and the updated-at trigger function and triggers. Preferences start with singular region, industry, and offering columns.	The base schema. Everything else builds on it.
`002_reference_offering_signals.sql`	Seeds signals for an earlier three-offering set, including a Concierto Cloud offering.	Superseded by 003. Not applied meaningfully on a fresh database after 003 runs.
`003_multi_value_preferences.sql`	Renames the preference columns to plural, converts them to text arrays, and adds non-empty checks and GIN indexes.	Lets a user choose several regions, industries, and offerings.
`003_update_offering_signals.sql`	Removes the older signals and seeds signals for the five Concierto offerings.	Sets the five-offering signal set.
`004_fix_offering_signals.sql`	Reinserts the five-offering signals.	Repeats 003's work, in case 003 was partly applied.
`004_preference_notification_email.sql`	Adds the notification email, backfilled from the user's email, and makes it required.	Digest recipient.
`005_force_upsert_offering_signals.sql`	Reinserts the five-offering signals again.	Safe to rerun. Repeats 003 and 004.

`006_schedule_frequency.sql`	Adds the schedule frequency, default daily.	Enables weekly and biweekly schedules.
`007_schedule_day.sql`	Adds the schedule day, default Monday.	Chooses the weekday for weekly and biweekly runs.

Things to know about the migration history

- Two files share prefix 003 and two share 004. They run in alphabetical order. This is safe because the later files only reinsert the same rows, but a new file must not reuse a prefix.
- The signal migrations repeat each other. On a fresh database the final state is the five-offering set. Do not treat the migration count as the number of distinct changes.
- Applied migrations must never be edited. The checksum check stops startup if one is changed. A fix is always a new numbered file.

The migration runner

`lib/db/migrate.ts` runs at startup and can also be run with `npm run db:migrate`.

42. Creates the `schema_migrations` table if it does not exist.
43. Takes a PostgreSQL advisory lock named for this project, so two instances cannot migrate at the same time.
44. Reads each file in `db/migrations/` whose name matches the pattern: digits, an underscore, lowercase letters, digits, and underscores, then `.sql`.
45. Computes the SHA-256 checksum of each file.
46. If the file is recorded as applied with the same checksum, it is skipped.
47. If the file is recorded as applied with a different checksum, the runner throws an error and stops. This protects the database from a silently edited migration.
48. Otherwise, it runs the file in a transaction and records the name and checksum.
49. Releases the lock.

If the runner fails, `instrumentation.ts` logs the error and the server still starts. A broken schema then appears as query errors at runtime, not as a failed deploy. Check the startup log after every deploy.

Local database

`docker-compose.yml` starts one PostgreSQL 16 (alpine) container with a named volume for data and port 5432 published to the host. The file sets the database user, password, and database name directly, which is acceptable for a local machine but should not be copied to a shared environment. Do not reuse those values in Azure.

13. Database Schema Reference

13.1 users

Column	Type	Notes
`id`	UUID, primary key	Stored in the session token.
`email`	Text, unique	The Azure AD sign-in email.
`name`	Text	Display name.
`role`	Text, default `Trianz employee`	Not checked anywhere in code.
`created_at`, `updated_at`	Timestamps	`updated_at` is set by trigger.

13.2 preferences

Column	Type	Notes
id	Primary key	
user_id	UUID, unique, foreign key to users	One preference set per user.
regions	Text array, not empty	Choices from the four region names.
industries	Text array, not empty	Free text, typically from the portal's list.
offerings	Text array, not empty	Choices from the five offerings.
notification_email	Text, required	Digest recipient.
schedule_time	Time	Local server time.
schedule_frequency	Text, default daily	daily, weekly, or biweekly.
schedule_day	Integer, default 1	0 (Sunday) to 6 (Saturday).
scheduled_discovery_enabled	Boolean, default true	Turns scheduled runs on or off. Manual runs are not affected.
workspace_name	Text, default Trianz GTM	Used in the digest subject.
updated_at	Timestamp	Maintained by trigger. Also the scheduler's baseline.

13.3 offering_signals

Column	Type	Notes
id	Primary key	
offering	Text	One of the five offerings.
signal	Text	The buying phrase, for example "Data center exit".
priority	Integer, 1 to 1000, default 100	Lower values run first in query generation.
is_active	Boolean	Only active signals are used.
created_at, updated_at	Timestamps	

Uniqueness: the offering and signal together are unique, and the offering with the lowercased signal is unique. This prevents the same phrase from being entered twice in different letter cases.

13.4 search_history

Column	Type	Notes
id	Primary key	
user_id	UUID, foreign key	The run's owner.
started_at, completed_at	Timestamps	completed_at is set only for finished runs.
articles_found	Integer	Articles returned by sources, before deduplication.
duplicates_removed	Integer	
leads_generated	Integer	Leads saved by this run.
status	Text	running, completed, or failed.
error_message	Text	Set on failure.
sources	Text array	Source names used.
search_queries	Text array	Query strings used.

A check constraint ties completed_at to a finished status, so a row cannot be half-complete.

13.5 articles

Column	Type	Notes
id	Primary key	
search_history_id	Foreign key, nullable	The run that first stored the article. Cleared if the run is deleted.
source	Text	Source name.
title, url, canonical_url, snippet	Text	Article details.
published_at, collected_at	Timestamps	
content_hash	Text, unique	The SHA-256 described in section 8.

13.6 leads

Column	Type	Notes
id	Primary key	
user_id	UUID, foreign key	The lead's owner.
article_id, search_history_id	Foreign keys, nullable	Cleared if the article or run is deleted.
company, initials, industry, country, signal, offering	Text	From the analyzer, with initials computed by the code.
source, source_url	Text	Publisher and article link.
score_label	Text	Very Hot, Hot, or Warm.
score_value	Integer, 0 to 100	
summary, reason	Text	Analyzer output.
project_stage, business_impact, procurement_activity	Text	Sales context from the analyzer.
industry_validated	Boolean	True for every saved lead.
content_hash	Text	Unique together with user_id.
published_at	Date	A date, not a timestamp.
detected_at	Timestamp	When the lead was saved.

Indexes support the common queries: the user with score, the user with detected date, the user with industry, and each foreign key.

13.7 Analyzer output (not stored as a table)

The model returns this structure. The code validates it before use.

Field	Type	Meaning
isValidOpportunity	Boolean	False rejects the article.
company	Text	The buyer.
industry	Text	The buyer's industry.
country	Text	Used for the region barrier.
signal	Text	The buying signal detected.
offering	One of five values	Used for the offering barrier.
summary	Text	One-line summary.
reason	Text	Why the lead qualifies.
projectStage, businessImpact, procurementActivity	Text	Sales context.
scoreLabel	Very Hot, Hot, or Warm	Score band.
scoreValue	Number, 0 to 100	Score.
industryValidated, regionValidated	Boolean	Validation flags.

`problemState`	A, B, or C	A: facing the problem. B: solving it. C: already solved.

14. API Reference

14.1 Route list

All routes require a session, except the NextAuth routes under `/api/auth/*`.

Method	Path	Purpose
GET	`/api/dashboard`	Lead counts, recent leads, latest run, and scheduling status.
GET	`/api/leads`	Filtered, paged list of the user's leads.
GET	`/api/leads/:id`	One of the user's leads.
GET	`/api/preferences`	The user's preferences.
POST	`/api/preferences`	Create or update the user's preferences.
POST	`/api/discover-leads`	Start a discovery run for the user now.
GET	`/api/search-history`	The user's recent runs.
GET	`/api/signals`	Signals, optionally filtered by offering.
POST	`/api/signals`	Add or update a signal.
GET	`/api/scheduler`	Scheduler status. Starts the timer if it is stopped.
POST	`/api/scheduler`	Run one scheduler check. `?force=true` runs every scheduled user.
GET	`/api/digest`	HTML digest of the user's recent leads.
GET	`/api/workspace`	The signed-in user's profile.
PATCH	`/api/workspace`	Update the signed-in user's name and role.
GET	`/api/healthz`	Health check.
GET, POST	`/api/auth/[...nextauth]`	NextAuth endpoints: sign-in, callback, session, sign-out.

14.2 Response envelope

Success:

```
{ "data": { } }
```

Error:

```
{ "error": { "code": "VALIDATION_ERROR", "message": "A readable explanation." } }
```

The HTTP status matches the error code. The digest route is the exception: it returns HTML, not JSON.

14.3 Request and response details

GET /api/leads

Parameter	Type	Rule
q	Text	Matches company or signal, case-insensitive.
industry	Text	Exact match.
country	Text	Exact match.
offering	Text	Exact match.
score	Text, may repeat	One or more of Very Hot, Hot, Warm.
limit	Integer	1 to 100.
offset	Integer	0 to 10000.

Response: `data.items` (the leads), `data.total`, `data.limit`, `data.offset`.

GET /api/leads/:id

Returns one lead. Returns `LEAD_NOT_FOUND` if the lead does not exist or belongs to another user.

GET and POST /api/preferences

The POST body accepts:

Field	Type	Notes
regions	Array of text	Not empty.
industries	Array of text	Not empty.
offerings	Array of text	Not empty.
notificationEmail	Text	Must be a valid address.
scheduleTime	Text, `HH:mm`	Local server time.
scheduledDiscoveryEnabled	Boolean	
workspaceName	Text	
scheduleFrequency	`daily`, `weekly`, or `biweekly`	Accepted by the code; not yet listed in the API contract document.
scheduleDay	Integer, 0 to 6	Accepted by the code; not yet listed in the API contract document.

GET returns `PREFERENCES_NOT_FOUND` if the user has no row.

POST /api/discover-leads

Starts a run for the signed-in user and returns the run summary: the run record, articles found, duplicates removed, and new articles stored. The request can take minutes, because each article is analyzed in turn. The client should expect a long wait and show progress from `GET /api/search-history`.

GET /api/search-history

Accepts `limit` from 1 to 100. Returns the user's runs, newest first.

GET /api/signals and POST /api/signals

- `GET` with no filter returns all signals. With `offering` values, it returns active signals for those offerings, sorted by priority.
- `POST` accepts `offering`, `signal`, and `priority` (1 to 1000). It upserts by offering and signal text, so posting the same pair updates the priority.

GET /api/scheduler, POST /api/scheduler

- `GET` returns `isRunning`, `lastTickAt`, `lastRunKey`, `activeSchedulesCount`, and `schedules`.
- `POST` returns a list of per-user results with a status of `triggered`, `skipped`, or `already_ran`. A triggered run that failed includes an error message.
- `POST ?force=true` returns `forced: true` and the results. It ignores time and frequency.

GET /api/digest

Returns HTML for the user's recent leads. The portal link inside the HTML defaults to `http://localhost:3000` unless it is set elsewhere, so set it before the digest is shared.

GET /api/workspace and PATCH /api/workspace

- `GET` returns the signed-in user.
- `PATCH` accepts `name` and `role`. Only the signed-in user is changed.

GET /api/healthz

Returns a success response for the App Service health probe.

14.4 Error codes

Code	HTTP	Meaning	Typical cause
`UNAUTHORIZED`	401	No valid session.	Session expired or not signed in.
`USER_NOT_FOUND`	404	The session refers to a missing user.	User row deleted while the session was active.
`PREFERENCES_NOT_FOUND`	404	No preferences saved.	New user who has not saved preferences.
`LEAD_NOT_FOUND`	404	No such lead for this user.	Wrong ID, or another user's lead.
`VALIDATION_ERROR`	400	Input failed validation.	Missing or out-of-range field.

Other codes are defined in `lib/http/response.ts`. Check that file before adding one.

15. Email and Digest

What the digest is

The digest is a short summary of the leads from one discovery run. It is designed to be read quickly: the top leads, their scores, and a link into the portal.

How it is sent

- After a run that produced leads, the discovery service calls the email service with the user's notification address, the leads, and the workspace name.
- The email service uses `EmailClient` from `@azure/communication-email` and the connection string in `AZURE_COMMUNICATION_CONNECTION_STRING`.
- The call is not awaited. The run returns its result before the email is finished.

When the send is skipped

Condition	Result
No connection string	Skipped, with a warning.
Invalid recipient address	Skipped.
No leads in this run	Skipped.

When the send fails

A failure is written to the console. The run is not affected, and nothing else records it. The team has no delivery status, no retry, and no alert. This is the main reason to treat digest email as best-effort.

Sender

The sender address is set in `email-service.ts`, not read from the environment. `AZURE_COMMUNICATION_SENDER_EMAIL` in `.env.example` is not used. Changing the sender needs a code change and a redeploy.

Power Automate and the HTML feed

`GET /api/digest` returns the same kind of content as HTML. Power Automate can call it on a schedule and forward the result. Two things to check before relying on it:

- The portal link inside the HTML defaults to `http://localhost:3000`. Set the correct portal address before the feed is shared.
- The route requires a session. Confirm how Power Automate authenticates, because a flow cannot sign in through a browser.

16. Setup and Local Run

Prerequisites

Requirement	Notes
Node.js 20 or 22 LTS	The project uses Next.js 16.
npm	Ships with Node.
PostgreSQL 16	Local, through Docker Compose, or Azure.
Docker Desktop	Optional, for the local database.
Trianz Azure AD app registration	Needed for sign-in. Ask the platform owner for the client ID and tenant.
Azure OpenAI access	Optional. Without it, the keyword fallback runs.
Azure Communication Services	Optional. Without it, the digest is skipped.

Steps

50. **Install.** Run `npm install` in the project folder.
51. **Start the database.** Run `docker compose up -d` to start the PostgreSQL container. Alternatively, point `DATABASE_URL` at any PostgreSQL 16 database.
52. **Create `.env.local`.** Copy `.env.example` and fill in the values. Section 17 lists every name.
53. **Apply migrations.** Run `npm run db:migrate`. The app also runs them at startup.
54. **Start the app.** Run `npm run dev` and open `http://localhost:3000`.

55. **Sign in.** Without the Azure AD values, sign-in cannot complete.
56. **Check the data.** Save preferences, then start a manual discovery run.

Verifying a local run

- The startup log shows the migrations finishing and the scheduler starting.
- `GET /api/healthz` returns a success response.
- `GET /api/scheduler` shows the timer running.
- A manual run completes and writes a `completed` row to `search_history`.

Common problems

Symptom	Likely cause
Sign-in loops back to the sign-in page	NEXTAUTH_URL or NEXTAUTH_SECRET missing or wrong.
Sign-in fails at the Microsoft step	Azure AD values missing or the redirect URI is not registered.
Runs complete with zero leads and no errors	Keyword fallback in use, or region and offering barriers rejecting everything. Check the run's articles and the region of the leads.
Migration error at startup	A migration file was edited after it was applied. Restore the original and add a new file.
Connection refused on the database	The container is not running, or DATABASE_URL points at the wrong port.

Scripts

Script	Purpose
`npm run dev`	Development server with Turbopack.
`npm run build`	Production build.
`npm start`	Production server.
`npm run typecheck`	Type check without writing output.
`npm test`	Unit tests.
`npm run test:db`	Migration tests. Needs a test database.
`npm run test:discovery`	Discovery tests.
`npm run test:ai`	AI analyzer tests.
`npm run test:e2e`	End-to-end script.
`npm run test:azure`	Live Azure OpenAI connection check.
`npm run db:migrate`	Apply migrations.

17. Environment Variables Reference

Only names are listed. Values go in `.env.local` locally and in App Service configuration in Azure. Never commit values, and never paste them into documents, tickets, or chat.

Database

Variable	Required	Default	Notes
DATABASE_URL	Yes	None	PostgreSQL connection URL.
DATABASE_SSL	Yes in Azure	On unless the host is local	Turns on SSL for the connection.
DATABASE_SSL_REJECT_UNAUTHORIZED	Yes in Azure	true	Keep true in production.
DATABASE_POOL_MAX	No	10	Maximum pooled

			connections.
DATABASE_IDLE_TIMEOUT_MS	No	30000	Idle connections close after this.
DATABASE_CONNECTION_TIMEOUT_MS	No	10000	Time to wait for a connection.
DATABASE_TEST_URL	Tests only	None	Test database. Never production.

Authentication

Variable	Required	Notes
AZURE_AD_CLIENT_ID	Yes	App registration client ID.
AZURE_AD_CLIENT_SECRET	Yes	Store in App Service settings or Key Vault.
AZURE_AD_TENANT_ID	Yes	Tenant ID.
NEXTAUTH_URL	Yes in production	Public address of the app. Missing from `.env.example`.
NEXTAUTH_SECRET	Yes in production	Signs session tokens. Missing from `.env.example`. Generate a long random value.

Azure OpenAI

Variable	Required	Default	Notes
AZURE_OPENAI_ENDPOINT	For live analysis	None	Resource endpoint.
AZURE_OPENAI_API_KEY	For live analysis	None	Without it, the keyword fallback runs.
AZURE_OPENAI_DEPLOYMENT	No	`gpt-4o`	Deployment name.
AZURE_OPENAI_API_VERSION	No	`2024-10-21`	API version.

Discovery

Variable	Status	Notes
RSS_SOURCES	Not read by the code	Listed in `.env.example`. The source list is set in `discovery-service.ts`.
GOOGLE_NEWS_LANGUAGE	Not read by the code	Listed in `.env.example`. The locale is hard-coded in the Google News adapter.

Email

Variable	Required	Notes
AZURE_COMMUNICATION_CONNECTION_STRING	For digest email	Without it, the digest is skipped.
AZURE_COMMUNICATION_SENDER_EMAIL	Not read by the code	Listed in `.env.example`. The sender is set in `email-service.ts`.

Obsolete

Variable	Notes
WORKSPACE_USER_EMAIL, WORKSPACE_USER_NAME, WORKSPACE_USER_ROLE	Replaced by the signed-in session user. Remove them from `.env.example` and from App Service.

18. Deployment and Azure Infrastructure

Production components

Component	Purpose	Notes
Azure App Service (Linux, Node)	Runs the Next.js app, the scheduler, and the API.	Run one instance. See section 10.
Azure Database for PostgreSQL Flexible Server	Application data.	SSL required.
Azure OpenAI, gpt-4o deployment	Article analysis.	Needs an endpoint and key in App Service settings.
Azure Communication Services	Digest email.	Needs a connection string.
Microsoft Entra ID app registration	Sign-in.	Redirect URI must include the app's callback path.

Resource names, hostnames, subscription, and resource group details are deliberately left out of this document. Ask the platform owner.

Deployment steps

57. **Create the App Service.** Linux, Node runtime, one instance.
58. **Set configuration.** Add every variable from section 17 that applies to production in App Service configuration. Keep secrets there, not in the repository.
59. **Register the redirect URI.** In the Azure AD app registration, add `<app-address>/api/auth/callback/azure-ad`.
60. **Deploy the code.** Use the team's usual pipeline or a manual deploy.
61. **Check startup.** The log should show the migrations completing and the scheduler starting. A migration error appears here, and the app still runs, so read the log.
62. **Verify.** Sign in, save preferences, and run a manual discovery. Confirm a `completed` row appears in `search_history`.
63. **Check the health probe.** `GET /api/healthz` should succeed.

Rollback

- Code rollback: redeploy the previous build. Migrations are not reversed automatically.
- Schema rollback: there is no down migration. A schema change must be fixed forward with a new numbered file.
- Scheduler rollback: stopping the app stops scheduled runs. Confirm this is acceptable before rolling back during business hours.

Company laptop setup

For a laptop behind a corporate network or proxy:

- Copy the repository without `node_modules`, `.next`, and `.env.local`. Create a new `.env.local`.
- Outbound HTTPS must reach Azure OpenAI, Azure Communication Services, Google News, PR Newswire, and Business Wire.
- A proxy that blocks the news sites produces zero articles and no error. Check the source warnings in the console.

- A proxy that blocks Azure OpenAI makes every article fail analysis. The fallback analyzer is not used in that case, so the run finishes with no leads.
- The full checklist is in `docs/COMPANY_LAPTOP_DEPLOYMENT.md`.

19. Developer Guide

Add a discovery source

64. Create a file in `lib/discovery/sources/` with a source object that returns collected articles for a query.
65. Add its name to the source type in `lib/discovery/sources/types.ts`.
66. Export it from `lib/discovery/sources/index.ts`.
67. Add it to the default list in `discovery-service.ts`.
68. Normalize each article's URL with the same function used for deduplication, so duplicates are caught.
69. Add a parser test under `tests/discovery/`, using a saved feed sample rather than a live request.
70. Run `npm test` and `npm run typecheck`.

Each new source adds one call per query per run, so the run time grows with the number of sources.

Add a buying signal

Signals live in the database, so no deploy is needed.

- Through the API: `POST /api/signals` with the offering, the signal text, and a priority from 1 to 1000.
- Through a migration: a new file with an insert that uses `ON CONFLICT` so it can be rerun.

Write signals in the same style as existing ones: short, specific phrases a news headline would contain. Vague phrases such as "cloud" produce noisy results. Use a low priority number for signals you want searched first. Remember that signals are shared across all users (section 5).

Add a migration

71. Create `db/migrations/NNN_description.sql`, using the next number and lowercase letters, digits, and underscores.
72. Write plain SQL. Use `IF NOT EXISTS` or `ON CONFLICT` where practical, so a rerun is safe.
73. Never edit a migration that has been applied anywhere. The checksum check will stop startup.
74. Run `npm run db:migrate` against a local database.
75. Run `npm run test:db` against a test database.
76. Update the schema sections of this document.

Add an API route

77. Create `app/api/<name>/route.ts`. Set the Node runtime and dynamic rendering.
78. Resolve the user with `getCurrentWorkspaceUser()`. Never accept a user ID from the request.
79. Validate input with the helpers in `lib/http/validation.ts`.
80. Return success and error responses with the helpers in `lib/http/response.ts`.
81. Add the route to section 14 and to `docs/api-contract.md`.
82. Add a test if the route has logic beyond passing data through.

Add an offering

An offering touches several parts of the system. Changing only the database is not enough.

83. The offering type in the AI client.
84. The system prompt in the AI client: the offering description and its customer pains.
85. The fallback analyzer's keywords and its branch for the offering.
86. The fallback query list in `discovery-service.ts`.
87. The signals, in a new migration.
88. The offering choices in the portal UI.
89. The tests that check the offering set.

Missing any of these means the offering either never matches or matches with the wrong wording.

Add a region or country

90. Add the country to the right list in `REGION_COUNTRIES` in `lib/domain/regions.ts`.
91. Add keyword matches if the model may return a variant or a city name.
92. Add a test that the country is kept for its region and rejected for others.

Layering rules

Layer	Location	Rule
Route handlers	`app/api/**/route.ts`	Authenticate, validate, call a service, return. No SQL.
Services	`lib/services/`	Business logic and orchestration. No direct SQL.
Repositories	`lib/repositories/`	The only place with SQL for each table.
Domain	`lib/domain/`	Types, mappers, and region data. No I/O.
Integrations	`lib/ai/`, `lib/discovery/`, `lib/services/email-service.ts`	Calls to external services.

Following these rules keeps the code testable. The services can be tested with fake repositories, and the integrations can be replaced with fakes in tests.

20. Codebase Structure

Top level

Path	Contents
`app/`	Pages and API route handlers.
`components/`	Portal components, including the main portal view.
`lib/`	All application logic (below).
`db/migrations/`	SQL migration files, applied in order.
`tests/`	Test files.
`docs/`	API contract, technical reference, this document, deployment notes, handoff notes, and an audit document.
`scripts/`	Command-line scripts, including the migration runner entry point.
`proxy.ts`	Authentication middleware for pages and API routes.
`instrumentation.ts`	Startup hook: migrations, then the scheduler.

`docker-compose.yml`	Local PostgreSQL container.
`.env.example`	Variable names with placeholder values.
`package.json`	Scripts and dependencies.

Inside lib/

Folder	Contents
`lib/auth/`	NextAuth configuration.
`lib/ai/`	Azure OpenAI client, prompt, and keyword fallback analyzer.
`lib/config/`	Reads and validates environment variables.
`lib/db/`	Connection pool and migration runner.
`lib/discovery/`	Source types, source adapters, and the RSS parser.
`lib/domain/`	Shared types, mappers, and region data.
`lib/http/`	Response and validation helpers.
`lib/repositories/`	SQL for each table.
`lib/services/`	Discovery, query generation, deduplication, lead generation, scheduler, and email.
`lib/workspace/`	Resolves the signed-in user for API calls.

Tests

File	Covers
`tests/ai/lead-generation.test.ts`	Analyzer output handling and lead filtering.
`tests/db/migrations.test.ts`	The migration runner, against a test database.
`tests/digest/power-automate.test.ts`	The digest content.
`tests/discovery/deduplication.test.ts`	URL normalization and content hashing.
`tests/discovery/discovery-service.test.ts`	The discovery run flow.
`tests/discovery/query-generation.test.ts`	Query strategies and the query cap.
`tests/scheduler/scheduler.test.ts`	Time matching, frequency rules, and the duplicate lock.
`tests/signals/signal-mapping.test.ts`	Signal mapping and filtering.
`tests/phase5-e2e.ts`	An end-to-end script, run separately.

21. Testing

Strategy

The tests are arranged in layers. Pure logic has unit tests that need no database or network. Stateful behavior, such as migrations, needs a real test database. Live integrations, such as Azure OpenAI and the news feeds, are checked by hand because their results change.

Unit tests

Run with `npm test`. These cover:

- Lead filtering in the AI and lead tests.
- URL normalization and content hashing.
- Query generation strategies and the query cap.
- The discovery run flow.
- Signal mapping and filtering.

- The digest content.

The README says there are 23 unit tests. The number changes as tests are added. Run `npm test` for the current count before quoting it.

Scheduler tests

`tests/scheduler/scheduler.test.ts` covers:

- Matching the scheduled time to the clock.
- The daily, weekly, and biweekly rules.
- The duplicate-run lock within one minute.

These tests show the rules, but they cannot test the restart behavior, because the lock is in memory and a test process starts fresh each time.

Migration tests

`tests/db/migrations.test.ts` runs the migration runner against `DATABASE_TEST_URL`. Run `npm run test:db`. The test is skipped when the variable is missing, so a green run without the variable does not prove the migrations work.

Warning: `DATABASE_TEST_URL` must never point at the production database, because the tests change schema.

End-to-end tests

`tests/phase5-e2e.ts` runs a script-level flow across the system. Run `npm run test:e2e`. It is not part of `npm test`, so it must be run on purpose.

Live checks

- `npm run test:azure` checks that the Azure OpenAI deployment answers. Run it after changing the endpoint, key, or deployment.
- Source feeds should be checked by hand after a change to a source adapter, because feed formats change without notice.

Not covered

- The Azure OpenAI analysis quality. Unit tests check the parsing and filtering, not whether the model judges well. Quality is judged by reading real leads.
- The real news feeds.
- Email delivery.
- The portal UI.
- The restart behavior of the scheduler.
- Concurrent runs for the same user.

What to run before a release

93. `npm run typecheck`
94. `npm test`
95. `npm run test:db` against a test database
96. `npm run build`
97. A manual run in a local or staging environment, checking that leads appear for the expected regions.

22. Known Issues and Gaps

22.1 Data and correctness

98. **A signal with an old offering name is dead.** Migration 002 seeded a Concierto Cloud signal and migration 003 removed it. Check that no active signal uses a name outside the five offerings, because such a signal can never produce a lead.
99. **Rejections are silent.** Offering, region, and industry rejections leave no record. Missing leads are hard to explain.
100. **Unknown countries are rejected.** Any country the list does not recognize maps to Global, which only passes for users who chose Global. A real opportunity in an unlisted country is lost for everyone else.
101. **Duplicate check covers 1,000 leads.** The company and offering check loads the user's 1,000 most recent leads. An older duplicate can become a second lead.
102. **Industry is not checked against the user's list.** The flag from the model decides. A lead in an industry the user did not choose is kept if the model says it is valid.
103. **Articles are analyzed one at a time.** Large runs are slow, and the request that started them may time out before the run finishes.
104. **Reuse is limited to 40 articles.** When nothing is new, only the first 40 collected items are checked against the current preferences.
105. **The fallback analyzer is not an AI model.** Results from runs without Azure credentials are not representative, and should not be used to judge lead quality.

22.2 Scheduler and operations

106. **Scheduled runs stop when the app stops.** Missed runs are not replayed.
107. **The duplicate lock resets on restart.** A restart within the scheduled minute can start a run a second time.
108. **Multiple instances run multiple timers.** Each instance starts its own runs. Keep one instance until a shared lock exists.
109. **The lock set grows without limit.** Keys are never removed, so the set gains one entry per scheduled user per day.
110. **The forced scheduler route ignores schedules.** `POST /api/scheduler?force=true` starts discovery for every scheduled user at once, with no confirmation.
111. **Digest sending is not tracked.** A failed send is only logged. There is no retry and no delivery status.
112. **Migration failure does not stop startup.** The app can start with an incomplete schema. Read the startup log after every deploy.
113. **Runs can stay "running".** If the process dies after a run starts and before the failure handler runs, the run row stays in the running state.

22.3 Security and access

114. **Signals are shared and unrestricted.** Any signed-in user can change signals, and the change affects every user with that offering.
115. **The scheduler triggers are open to any signed-in user.** Any user can force a run for every scheduled user.
116. **No role enforcement.** The role field is stored but never checked.
117. **Local database credentials are in the Compose file.** `docker-compose.yml` sets a fixed user and password. That is acceptable only on a local machine.
118. **Auth variables are missing from `.env.example`.** `NEXTAUTH_URL` and `NEXTAUTH_SECRET` are not listed, so a new setup can fail in a confusing way.

119. **Azure AD values are not checked at startup.** A missing value shows up at the first sign-in.
120. **External callers need a session.** It is not clear how Power Automate authenticates to the scheduler and digest routes.

22.4 Configuration and drift

121. **Unused variables in `.env.example`.** `RSS_SOURCES`, `GOOGLE_NEWS_LANGUAGE`, `AZURE_COMMUNICATION_SENDER_EMAIL`, and the `WORKSPACE_USER_*` variables are not read by the code.
122. **The sender address is hard-coded.** The email sender is set in code, so changing it needs a redeploy.
123. **The digest link defaults to localhost.** Set the portal address before the digest is shared.
124. **The README and handoff are out of date.** The README says to apply migrations 001 to 004; the repo has 001 to 007. Both describe three offerings; the code has five. Both describe Google News as the only source; the code uses three. Both describe the placeholder workspace user; the app uses Azure AD sign-in.
125. **The API contract omits two fields.** `scheduleFrequency` and `scheduleDay` are accepted by the code but not listed in `docs/api-contract.md`.
126. **Test count is unverified.** The README says 23 unit tests. Verify with `npm test`.
127. **Query format is described incorrectly elsewhere.** Older notes show a simple signal, industry, and region query. The real queries use the four strategies in section 8.

23. What's Left to Do

Listed in rough priority order.

Reliability

128. Store scheduler runs in the database, and use a database lock, so schedules survive restarts and work with more than one instance.
129. Await or retry the digest send, and record its delivery status.
130. Mark stale "running" runs as failed after a timeout.
131. Analyze articles with a concurrency limit, instead of one at a time.

Visibility

132. Record a reason for every rejected article, so missing leads can be explained without rerunning the model.
133. Show unknown countries to the user, rather than mapping them to Global.
134. Log a summary per run: queries run, articles per source, duplicates, rejections by reason.

Access control

135. Add an admin role, set from an allow-list of Azure AD users, for signal changes and scheduler triggers.
136. Restrict the forced scheduler route to admins, and require confirmation in the portal.
137. Document and implement the authentication path for external triggers.

Accuracy

138. Check the industry against the user's chosen industries, instead of relying only on the model's flag.
139. Extend the duplicate check beyond the 1,000 most recent leads.
140. Test the analyzer against a set of labeled articles, and track its accuracy over time.

Configuration and documentation

141. Read the sender address and the source list from configuration, and remove unused variables.
142. Add `NEXTAUTH_URL` and `NEXTAUTH_SECRET` to `.env.example`, with placeholder values.
143. Update the README, the handoff, and the API contract to match the code: seven migrations, five offerings, three sources, Azure AD sign-in, and the two undocumented fields.
144. Set the portal address for the digest link through configuration.

24. Glossary

Term	Meaning
Lead	A qualified enterprise opportunity for one user, saved from one article.
Signal	A buying-intent phrase tied to an offering, such as "data center exit". Shared across users.
Offering	One of the five Concierto products the user can monitor.
Region	APAC, EMEA, North America, or Global. Each region has a fixed country list.
Score	Very Hot (85 to 100), Hot (70 to 84), or Warm (50 to 69), with the number stored alongside the label.
Problem state	A: the buyer faces the problem. B: the buyer is solving it. C: the buyer has already solved it, so the lead is redirected to a later offering.
Discovery run	One pass of query, fetch, deduplicate, analyze, and save. Recorded in search history.
Content hash	SHA-256 of the normalized title, canonical URL, and snippet. Detects duplicate articles and leads.
Canonical URL	An article URL with tracking parameters removed and remaining parameters sorted.
Offering barrier	The rule that a lead is kept only for an offering the user chose.
Region barrier	The rule that a lead is kept only for a country in one of the user's regions.
Fallback analyzer	The keyword-based analyzer, used when Azure OpenAI is not configured.
Digest	The summary email, and the matching HTML feed, of a run's new leads.
Fail-open	When the scheduler's last-run check fails, the run goes ahead rather than being skipped.
Baseline reset	The rule that saving preferences restarts the schedule cadence.
Advisory lock	A PostgreSQL lock used to stop two instances from migrating at the same time.
Envelope	The `data` or `error` wrapper around every JSON API response.

25. Summary

What the system does

The GTM Lead Intelligence Agent watches public news and press releases for enterprise buying signals, scores each opportunity with an AI model, and delivers region-filtered leads to Trianz sales teams. It runs as one Next.js application on Azure App Service, with PostgreSQL for storage, Azure OpenAI for analysis, Microsoft sign-in, and Azure Communication Services for email.

How it works

Signals and preferences become search queries. Articles are collected from three sources, normalized, and deduplicated. Each new article is analyzed by the model and checked against the user's offerings and regions before it is saved. Runs are recorded, dashboard counts update, and a digest is sent.

Strengths

- Signals are data, not code, so sales owners can change what is searched with a database change.
- Two checks after the model make the offering and region rules reliable, even when the model is not.
- Leads are private to each user, and every query is scoped to the signed-in user.
- Migrations are versioned and checksum-checked, so the schema history is verifiable.

Risks

- Scheduled runs depend on a single process that must stay up, and the duplicate lock does not survive a restart.
- Signals are shared and open to any signed-in user.
- Rejections are not recorded, so the rules cannot be audited from the database.
- Digest delivery is not tracked.
- The documentation has drifted from the code in several places.

What comes next

Section 23 lists the work in priority order. The first items are durable scheduling, a record of rejections, delivery tracking for the digest, and an admin role for signal changes. Each one reduces a risk listed above.